# One Tile, Two True Colors

Quantum contextuality as a design material

Lakshmi Monish

October 2026
github.com/QubitMan-hub/Qontext



**Abstract.** Design systems give every token one value. Quantum mechanics has a theorem saying some quantities cannot have one: the Mermin-Peres square, a three by three grid of two-qubit measurements whose rows and columns obey six product rules that no fixed table of $\pm 1$ values can meet. Qontext turns the square into a color system. Each of its nine tiles holds two colors, one when it is read with its row and one when it is read with its column, and hue plays the role of quantum phase. We show that the two readings always differ on an odd number of tiles, and on 3 to 4 tiles on average for any pure state. We describe a contrast checker that tests text against every color a tile can take, a theme dial built from an interferometer model, and a case study in which the nine panels of a sneaker are the nine tiles. On IBM's `ibm_kingston` each of the six rules held on 93 to 95% of shots, for a score of 5.29 against the classical limit of 4. Everything else runs as exact simulation in a web browser, and we claim no quantum advantage.

| | |
|---|---|
| **The idea** | A color token may have two true values, one per context, with a proof that no single palette can stand in for it. |
| **What we show** | The two readings of the grid differ on an odd number of tiles, never zero, and on 3 to 4 on average (Results 1 and 2). On IBM hardware the square scores 5.29 against a classical limit of 4 (Section 8). |
| **What it is not** | A speed-up. Two qubits are easy to simulate on any laptop; what the physics contributes is a constraint on palettes that can be proved (Section 10). |

## 1 Introduction

A design token is a named value: `color-accent` is `#3fc0f2`, everywhere, for everyone. Modern design systems do let tokens change with context. A token can have a light mode and a dark mode, or a hover state. But in every case a designer picks the value for each context by hand, and nothing stops them from picking one value for all of them. The context is a label.

Quantum mechanics offers a different kind of context. In 1967 Kochen and Specker showed that there are sets of measurements whose outcomes cannot all be fixed in advance, independent of which other compatible measurements are made alongside them [1, 2]. In 1990 Peres and Mermin found a version small enough to write on a napkin: a three by three grid of two-qubit observables, now called the Mermin-Peres square [3, 4]. Each row and each column is a set of observables that can be measured together, and quantum mechanics fixes the product of their outcomes. No assignment of $+1$ and $-1$ to the nine cells can satisfy all six products at once, so no single table of values can stand behind every

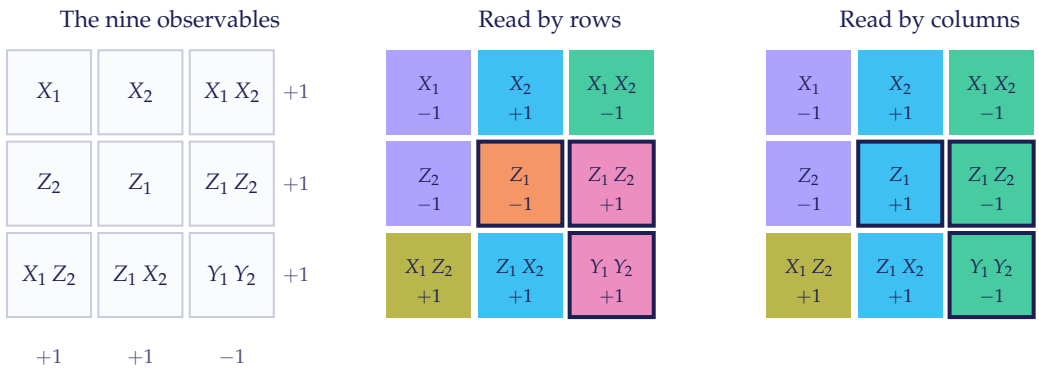



**Figure 1.** One tile, two true colors. World 4,227,792 with a triad ($d = 3$). Left: the nine observables. Middle and right: the same measurement read by rows and by columns. Outlined tiles differ between the readings. The numbers beside and below the first grid are the products each row and column must have.

row and every column.

Qontext turns this into color (Figure 1). Each cell becomes a tile. Each outcome becomes a hue: $+1$ shows the tile's harmony color, $-1$ its complement. Reading the grid by rows gives one coloring, reading it by columns gives another, and the mathematics guarantees that the two can never be the same coloring. Here the context does real work: it decides which color a tile shows, and no fixed palette can reproduce both readings.

We built this as a working product, not a demonstration: a website where every visitor gets their own measured palette, a studio for designers (the Atelier), a 12 KB runtime that applies a measured palette to any website, a Figma plugin with Row and Column variable modes, and a command line tool that runs the same circuits on IBM Quantum hardware. This paper describes the ideas and their limits. Its contributions are:

- A mapping from the Mermin-Peres square to color tokens, with hue as quantum phase (Section 3).
- Two short results about the two readings: they differ on an odd number of tiles, and on $\frac{1}{2}\sum_t(1 - \langle O_t\rangle^2)$ tiles on average, which is between 3 and 4 for any pure state (Section 4).
- Bouncer, a contrast check over every color a tile can take, and the Phase Dial, a light and dark theme built as an interferometer (Section 5).
- Pair Production, a worked case of an artist designing a sneaker drop with the system (Section 6).
- An implementation in which every classical limit shown to the user is checked by exhaustive search in the test suite (Section 7).
- A run on IBM's `ibm_kingston`, where every rule of the square held on at least 93% of shots (Section 8).

One caveat runs through the whole paper. Two qubits are easy to simulate, so nothing here needs a quantum computer; the hardware run in Section 8 adds physical randomness and a provenance record, not capability (Section 10).

| | column 1 | column 2 | column 3 | row product |
|---|---|---|---|---|
| row 1 | $X_1$ | $X_2$ | $X_1X_2$ | $+I$ |
| row 2 | $Z_2$ | $Z_1$ | $Z_1Z_2$ | $+I$ |
| row 3 | $X_1Z_2$ | $Z_1X_2$ | $Y_1Y_2$ | $+I$ |
| column product | $+I$ | $+I$ | $-I$ | |

**Table 1.** The Mermin-Peres square as Qontext uses it. Rows and columns are the six contexts.

| Structure | classical limit | quantum value | checked over |
|---|---|---|---|
| Mermin-Peres score ($2k-6$ for $k$ rules held) | 4 | 6 | all 512 tables |
| KCBS sum of probabilities | 2 | $\sqrt{5} \approx 2.236$ | all 32 |
| CHSH value $S$ | 2 | $2\sqrt{2} \approx 2.828$ | all 16 strategies |
| Yu-Oh expression | 8 | $25/3 \approx 8.33$ | all 8,192 |
| Peres 33, Cabello 18 coloring | impossible | | complete search |

**Table 2.** Classical limits and quantum values shown in Qontext. The Yu-Oh set can be colored, but its expression still separates the two.

## 2 Background

**Contextuality.** Two observables are compatible when they commute, so they can be measured together. A *context* is a set of mutually compatible observables. A noncontextual hidden variable model gives every observable one predetermined value, the same in every context it appears in. The Kochen-Specker theorem says no such model can reproduce quantum mechanics in dimension three or more [1]. A recent review covers the theory and the experiments in depth [5].

**The Mermin-Peres square.** Table 1 shows the version we use. $X_i$, $Y_i$ and $Z_i$ are Pauli operators on qubit $i$. The three observables in each row commute, and so do the three in each column. Each row multiplies to the identity, as do the first two columns. The third column multiplies to minus the identity, because $X_1X_2 \cdot Z_1Z_2 = -Y_1Y_2$. Our engine derives these six signs from the matrices when it loads, rather than storing them.

Suppose each cell had a fixed value $v_t \in \{+1, -1\}$. Multiplying all nine values row by row gives $+1$, since each row gives $+1$. Multiplying them column by column gives $-1$. It is the same nine numbers, so this is a contradiction, and at least one of the six rules must fail. Checking all $2^9 = 512$ tables confirms that the best any table can do is five rules out of six. Quantum mechanics satisfies all six on every run, whatever the state, which makes the square a state-independent test. It has been measured with trapped ions [6], on IBM's public quantum computers [7], and as a two-player game with entangled photons [8].

**Other structures.** The engine also builds the KCBS pentagram [9], the CHSH setting [10, 11], and three Kochen-Specker sets: Peres's 33 rays [12], the 18 vectors of Cabello, Estebaranz and García-Alcaine [13], and the 13 rays of Yu and Oh [14]. Table 2 lists the classical and quantum values the interface shows. Each classical value in the table is checked by exhaustive search in the test suite, not copied from the literature.

## 3 From observables to colors

**Hue as phase.** A quantum system with $d$ levels has a clock operator whose eigenvalues are the $d$-th roots of unity, $\omega^k = e^{2\pi ik/d}$. They sit evenly on a circle. We place them on the

hue circle of OKLCH, the cylindrical form of the Oklab color space [15, 16], so a palette with base hue $h_0$ has the hues

$$h_k = h_0 + \frac{360^\circ k}{d}, \qquad k = 0, \ldots, d-1, \tag{1}$$

all at one lightness $L$ and chroma $C$. With $d = 2$ this is a complementary pair, with $d = 3$ a triad, with $d = 4$ a square, up to $d = 8$. These are the classic even harmonies of color theory [17]. The analogy is structural: the eye does not see quantum phase, but the same arithmetic of turning by equal steps produces both.

**Tile colors.** Tile $(r, c)$, with $r, c \in \{0, 1, 2\}$, takes harmony color $k = (3r + c) \bmod d$. An outcome of $+1$ shows hue $h_k$. An outcome of $-1$ shows $h_k + 180^\circ$, its complement. A three by three grid therefore has eighteen tile colors, two per tile, drawn from at most $2d$ distinct hues.

**Measurement.** The engine samples outcomes exactly. For a context with observables $A$ and $B$ and third observable $C = sAB$ (with $s = \pm 1$ from Table 1), the probability of outcomes $(a, b)$ on state $|\psi\rangle$ is

$$P(a, b) = \left\| \tfrac{I+aA}{2} \, \tfrac{I+bB}{2} |\psi\rangle \right\|^2, \qquad c = s\,a\,b. \tag{2}$$

Each of the six contexts is measured on its own copy of $|\psi\rangle$, as in the experiments above. The random numbers come from a seeded generator, so a measurement is reproducible from two numbers: the state's seed and the measurement's seed. On the website each visitor gets a measurement seed between 1 and 9,999,999, called their *world*, and the whole site is colored from it. The state is a fixed random two-qubit state (Gaussian amplitudes, normalized). A design, with its harmony, lightness, chroma and seeds, travels as a short code beginning `QX1.` that any part of the system can open.

## 4 Two true colors

Reading the grid by rows takes each tile's value from its row's measurement. Reading it by columns takes each tile's value from its column's measurement. Figure 1 shows both readings for one world. Three tiles differ.

For a designer, two questions follow: can the readings ever agree, and how much do they usually differ? The answers are short.

**Result 1: Always an odd number.** *In every measurement, the row reading and the column reading differ on an odd number of tiles: 1, 3, 5, 7 or 9. They never agree everywhere.*

*Proof.* Let $R_t$ and $C_t$ be tile $t$'s values in the two readings. Every row satisfies its rule, so $\prod_t R_t = +1$. Every column satisfies its rule, so $\prod_t C_t = -1$. Hence $\prod_t R_t C_t = -1$. A tile where the readings agree contributes $+1$ to this product and a tile where they differ contributes $-1$, so the number of differing tiles is odd. □

This is the square's contradiction seen from the designer's side: there is always at least one tile with two true colors.

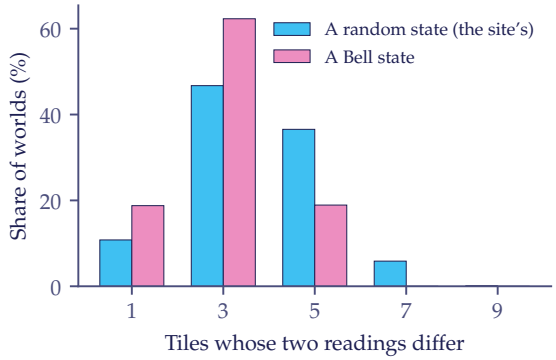



**Figure 2.** How many tiles differ between the two readings, over 100,000 worlds. Only odd counts occur.

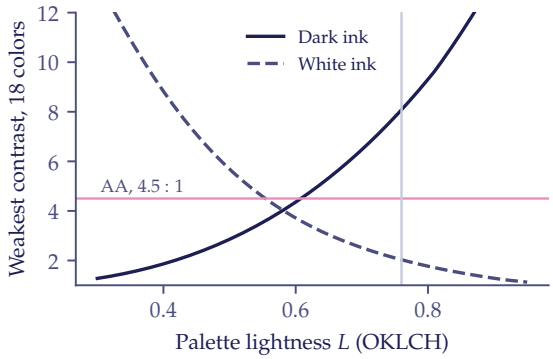



**Figure 3.** Bouncer. The weakest contrast of each ink over all eighteen tile colors, as the palette's lightness moves. The grey line is the site's lightness.

**Result 2: Three to four on average.** *On a state where tile $t$ has expectation $\langle O_t \rangle$, the expected number of differing tiles is*

$$\mathbb{E}[\text{differing tiles}] = \frac{1}{2} \sum_{t=1}^{9} \left(1 - \langle O_t \rangle^2\right). \tag{3}$$

*For any pure two-qubit state this lies between 3 and 4, and both ends are reached.*

*Proof.* A tile's two values come from two separate runs on the same state, so they are independent draws of the same observable $O_t$. Each is $+1$ with probability $p_t = (1 + \langle O_t \rangle)/2$, so they differ with probability $2p_t(1 - p_t) = (1 - \langle O_t \rangle^2)/2$. Linearity of expectation gives the formula.

For the bounds: for a pure two-qubit state the squared expectations of the fifteen non-identity Pauli operators sum to 3, because $\operatorname{tr} \rho^2 = 1$. The nine tiles are nine of those fifteen, so $\sum_t \langle O_t \rangle^2 \leq 3$. The other six split into two triples of operators that all anticommute with each other, $\{Y_1, X_1Y_2, Z_1Y_2\}$ and $\{Y_2, Y_1X_2, Y_1Z_2\}$. Within such a triple the squared expectations sum to at most 1: for any unit vector $c$ the operator $\sum_i c_i P_i$ squares to the identity, so its expectation is at most 1, and we choose $c$ along the vector of expectations. So the six hold at most 2, the nine hold at least 1, and the expected count lies between 3 and 4.

The top end is reached by $|0\rangle \otimes |{+i}\rangle$, where only $Z_1$ among the tiles has a definite value. The lower end is reached when all of the state's Pauli weight lies on the nine tiles. A Bell state is one example: it fixes $X_1X_2$, $Z_1Z_2$ and $Y_1Y_2$. The product state $|00\rangle$ is another: it fixes $Z_1$, $Z_2$ and $Z_1Z_2$. □

A fixed tile reads the same in both contexts, so for these states at most five tiles can differ.

Figure 2 checks both results against the engine over 100,000 worlds. Only odd counts occur. For the site's random state the formula predicts 3.752 differing tiles and the simulation gives 3.754. For the Bell state the formula gives exactly 3 and the simulation 3.003. The state is therefore a design choice: a state that fixes some tiles gives calmer worlds, with fewer tiles that change between readings.

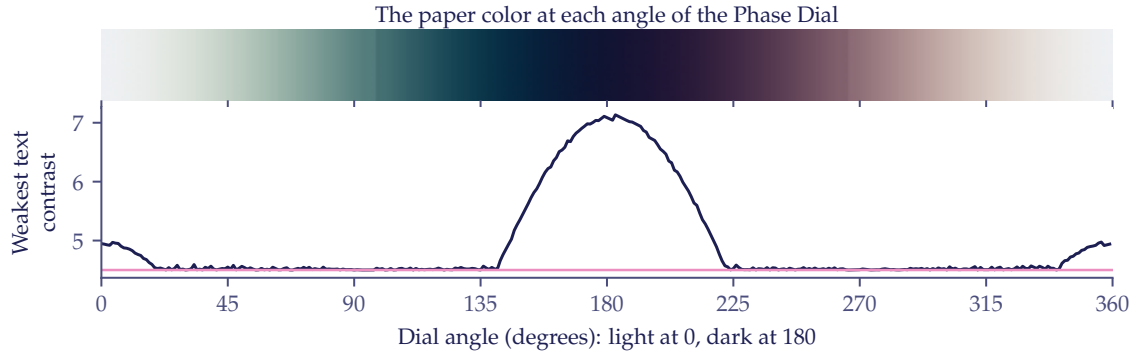



**Figure 4.** The Phase Dial. Top: the page's paper color at each angle. Bottom: the weakest contrast of any text color on any background at that angle. It never drops below the line at 4.5 : 1.

## 5 Making it usable

A palette that changes with context is only useful if text stays readable on every color it can show. Two tools handle this.

**Bouncer.** Text on a tile must read on both of the tile's colors, since the reader may see either. Bouncer computes the contrast of an ink against all eighteen tile colors, using the WCAG 2 contrast ratio [18] and the APCA lightness contrast [19], and reports the weakest. A palette whose weakest ratio is below 3 : 1 cannot be exported. Between 3 and 4.5 it is marked as fit for large text only. When a palette fails, Bouncer offers the nearest lightness that passes, searched in steps of 0.01. Figure 3 shows the trade-off for one triad: dark ink passes once $L \geq 0.61$, white ink while $L \leq 0.55$, and between the two there is a band where neither passes on all eighteen colors.

**The Phase Dial.** The site's light and dark themes are two points on one turn. A theme at dial angle $\varphi$ passes each color token through the model of a Mach-Zehnder interferometer, the same model the system's color mixer uses. The light value enters one arm and the dark value the other. Lightness follows the intensity at one output port,

$$L(\varphi) = \cos^2\tfrac{\varphi}{2}\, L_{\text{light}} + \sin^2\tfrac{\varphi}{2}\, L_{\text{dark}}, \tag{4}$$

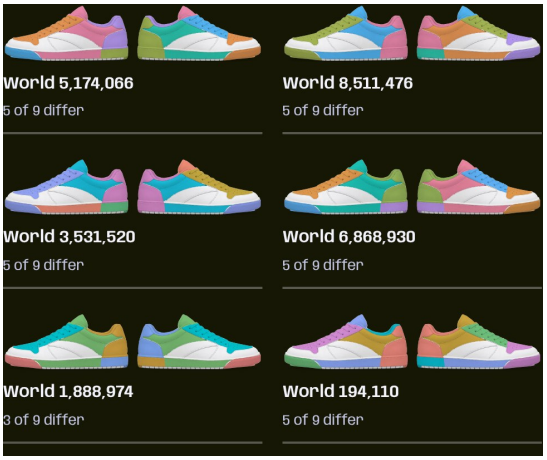
and the two chromas add as waves with relative phase $\varphi - 180^\circ$. Because of that cross term, the way from light to dark passes a teal dusk and the way back a plum one. Every whole degree is a usable theme. Bouncer then moves the background walls out of the band where neither black nor white text reads, and moves each text color's lightness until it reaches 4.5 : 1 on every wall. Figure 4 shows the paper color and the weakest text contrast at all 360 angles. The minimum is 4.50, reached near 93°. As with hue, the interferometer is a model for interpolation, not a claim about optics.

## 6 Case study: Pair Production

An engine is easier to judge in use. Pair Production is a section of the Qontext website that follows an artist through a sneaker drop, from first color to finished pair. Sneakers suit the idea unusually well: a drop is a limited run, collectors care about which run a pair came from, and a pair is two objects that are supposed to match, which is exactly the expectation the square breaks.




**(a)** The pair. The left foot reads the grid by rows, the right foot by columns. The stripe, heel counter and heel sole differ: the three outlined tiles of Figure 1.




**(b)** Six of the drop's twelve pairs, each from its own world and labeled with how many panels differ between its feet. Every count is odd.

**Figure 5.** Pair Production for world 4,227,792.

**The mapping.** A low-top sneaker is divided into nine panels in three bands and three zones. The bands are the rows: the top band (laces, tongue, collar), the body band (toe box, stripe, heel counter) and the sole band (toe bumper, midsole, heel sole). The zones are the columns: toe, middle and heel. The left foot shows the row reading and the right foot shows the column reading. The two feet of a pair therefore differ on exactly the tiles where the readings differ, which by Section 4 is always an odd number of panels. Figure 5a shows the pair for the world of Figure 1. The stripe, heel counter and heel sole differ between the feet. The other six panels match.

**The workflow.** The section is a scroll story in six steps beside a pinned display case.

1. *Roll your world.* The visitor's world decides which panels can change between the feet. Rolling a new one recolors the whole site.
2. *Tune the harmony.* Turning the base hue and choosing $d$ changes every panel's pair of colors at once.
3. *Bouncer checks the print.* Anything printed on the shoe has to read on all eighteen panel colors. The colors are laid over a photograph of white leather with a multiply blend, so each panel is a little darker than its color. Bouncer multiplies each color by the measured brightness of the leather under that panel (between 0.78 and 0.93) before it checks, and offers the nearest passing lightness when the check fails.
4. *Paint the pair.* The visitor taps a panel to measure it. It lands on its row color on the left foot and its column color on the right. A button tries to force the feet to match. The six rules are listed in shoe terms, and a matching pair always breaks at least one of them, because at most five of six can hold.
5. *The drop.* Twelve more pairs from twelve other worlds, each following all six rules (Figure 5b). Choosing one makes it the visitor's world.



**Figure 6.** The same pair on the Turntable. The panel colors are projected from the photograph's outlines onto a 3D model generated from the photograph. The left foot is the right foot mirrored.

6. *The tag.* A hang tag records the world, the rules held and the panels that differ, with a SHA-256 fingerprint of the design code so anyone can check a pair is the one that was made. The visitor can download a paint sheet with every panel's color, open the design in the Atelier, or put the pair on the Turntable.

**The shoe.** The shoe is a studio photograph of a plain white sneaker, made with an image generator. A script finds the seams as thin dark lines, splits the leather into regions between them, and assigns the regions to the nine panels, with a few hand-drawn cuts where a seam is only partly visible. Each panel becomes a vector outline. The browser lays each panel's color over the photograph with a multiply blend, so the leather's shading, stitching and laces show through every color. The quarter and mudguard stay white, as on a real colorway.

**The Turntable.** The final step shows the pair in three dimensions (Figure 6). The model was generated from the same photograph with an image-to-3D model (Meshy). A script fits the model's side outline to the photograph's outline by maximizing their overlap, which reaches an intersection over union of 0.965, and stands the model in the photograph's frame. In the browser, the nine panel outlines are painted on a flat image the size of the photograph and projected through the shoe from the side, so a point at model position $(x, y)$ takes the color of the photograph's pixel at the matching place. The lining and insole, found from the direction their surfaces face, stay white. Because the 2D and 3D views share one set of outlines, they cannot disagree about which panel is which.

## 7 Implementation and verification

The engine is a set of plain JavaScript modules with no dependencies: color science, a small linear algebra library, the quantum structures above, and exporters for CSS variables, W3C design tokens [20], Tailwind and SVG. The website, the Atelier, the runtime and the Figma plugin are built from it, and they share one convention for turning seeds into measurements, so a world looks the same everywhere.

Every classical limit the interface shows has a test that finds it by exhaustive search (Table 2). The unit tests (92 at the time of writing) cover the engine and runtime. The browser tests (145) drive the website and the Atelier in Chromium: they check, for example, that the pair's two feet differ exactly on the contextual tiles, that forcing a match breaks a

rule, that the Turntable shows the same colors as the 2D pair, and that the pages have no accessibility violations under axe-core at phone and desktop sizes in light and dark. A separate check holds the website to a performance budget on a throttled phone profile.

For hardware, a command line tool builds each row and column as a circuit in which every observable is measured with its own ancilla qubit. The three values of a context are therefore measured separately, so noise can break a rule, as it does on real devices. In the tool's noisy simulation the Mermin score is about 4.7, between the classical limit of 4 and the ideal 6. On IBM's public devices, Dikme et al. measured a comparable violation using a single ancilla per context [7]. Recorded hardware shots can be saved as a pool and replayed by the Atelier in place of the simulator, which gives physically random worlds and a job record that serves as provenance.

## 8 A run on IBM hardware

We ran the tool on IBM's `ibm_kingston` on 12 October 2026, using the free Open Plan. Each job held ten circuits, the six contexts of the square and the four CHSH settings, prepared on the same seeded random state the website uses. We made three runs; the first two exposed a qubit-placement problem described below, and we report the third, at 2,048 shots per circuit or 20,480 measurements in all, in Table 3.

| Context | row 1 | row 2 | row 3 | column 1 | column 2 | column 3 |
|---|---|---|---|---|---|---|
| Rule held | 93.8% | 92.9% | 95.3% | 93.8% | 93.4% | 95.0% |

| Score | classical limit | ideal quantum | measured |
|---|---|---|---|
| Mermin-Peres score | 4 | 6 | $5.286 \pm 0.026$ |
| CHSH value $S$ | 2 | $2\sqrt{2} \approx 2.828$ | $2.732 \pm 0.032$ |

**Table 3.** The run on `ibm_kingston`, job `db6cbpslf4us73c54gk0`. Each rule held on 93 to 95% of its 2,048 shots. Errors are one standard error from the shot counts.

Both scores lie far above their classical limits, the Mermin-Peres score by 50 standard errors and the CHSH value by 23. Averaged over the six contexts, 5.64 of the six rules held per round, while any model with fixed values keeps at most five on average. This is what a working device should do, and we make no claim beyond it; the point for Qontext is that colors drawn from these shots come from a run that, taken as a whole, exceeded the classical limit.

The first two runs went wrong in an instructive way. At Qiskit's light optimization level the transpiler placed the two-qubit CHSH circuits on the first two qubits of the chip whenever they fitted, without checking their error rates. On `ibm_kingston` one of those qubits misread 17% of its results, and the CHSH value came out at 1.11 and then 1.05, below the classical limit, while the Mermin-Peres circuits, which the transpiler had already placed with care, scored 5.15 and 5.26. IBM's simulated copy of the device reproduced the drop, with a CHSH value of 1.15. With a noise-aware layout for every circuit the CHSH value rose to 2.73, and a test on the simulated device now catches the mistake.

The shots are stored in the repository as `data/hardware/ibm_kingston-2026-10-12.json`. The Atelier ships the same run and stamps it. The SHA-256 fingerprint of its shot data is

`91bf9f705b18ad2a72a0c4ca67c0ebce5181454d32af786592c78e51e45dcdd0`

so anyone can check that the numbers above come from those shots.

## 9 Related work

**Quantum computing and art.** Artists and researchers have used quantum computers to make music [21], paintings [22] and game content. Wootton's quantum blur encodes an image in a quantum state and lets interference blur it [23]. Quantum Brush offers painting brushes that turn strokes into quantum algorithms and map the measured states back to color [24]. Earlier still, WaveFunctionCollapse borrowed the language of measurement for procedural generation [25]. These works use quantum dynamics or randomness as a source of new effects. Qontext uses a different part of the theory. It takes a no-go theorem about values and makes it a rule of the design system itself: a token is allowed, and in fact required, to have two values, and no fixed palette can imitate it. We share Wootton's view that the useful thing at this scale is designing software with quantum principles, not running it on quantum hardware [23].

**Contextuality.** The theory and its experiments are reviewed by Budroni et al. [5]. The Mermin-Peres square has been tested with ions [6], on public cloud devices [7] and as a nonlocal game with photons [8]. Our results about differing tiles are elementary consequences of the square; we state them because they describe what a designer actually sees on the page.

**Color and accessibility.** Qontext works in OKLCH [15, 16], where equal steps in hue and lightness look more even than in older spaces. Contrast follows WCAG 2.2 [18], with APCA [19] reported alongside.

## 10 Limits

**No quantum advantage.** Two qubits live in a four-dimensional state space, and the browser simulates them exactly in microseconds. A classical program that is told which context is being read can reproduce every reading, which is precisely what our simulator does: the theorem rules out fixed tables of colors, not software. What Qontext offers instead is a design rule with a proof behind it, namely that any fixed palette placed on the grid breaks at least one of the six rules. Real hardware contributes physically random collapses, measured violations of classical limits and a job record, and nothing beyond those three.

**The hardware test.** Like other contextuality experiments on noisy devices, ours assumes that the three measurements in a context are compatible. Gate and readout errors mean they are only approximately so, which is the compatibility loophole discussed by Budroni et al. [5]; the run is a demonstration on a working device, not a loophole-free test. The CHSH circuits use two neighbouring qubits on one chip, so their value says nothing about nonlocality.

**Randomness.** Worlds on the website come from a seeded pseudorandom generator, chosen so that a world can be shared and reproduced. They are physically random only when replayed from a hardware pool.

**Perception.** Hue as phase is an analogy that produces even harmonies. Whether contextual palettes help designers, or confuse readers who see a color change between two places, is an open question. We have not run a user study. Bouncer makes sure text stays readable, but readability is not the same as clarity.

**Contrast metrics.** The WCAG 2 ratio is known to misjudge some dark color pairs, and APCA is still a draft. Bouncer's leather model, a single brightness factor per panel, is an approximation of a printed surface.

**The shoe.** The panel outlines were finished by hand where seams were faint. The 3D model was generated from one photograph, so its far side is the generator's guess. Projecting the outlines from the side is exact on the sides of the shoe and approximate on its top, where the panel borders are the side view's borders.

## 11 Conclusion

A design token with one value behaves like a noncontextual hidden variable. The Mermin-Peres square is a small, precise case where no such value can exist, and we have turned it into a working color system: tiles with two true colors, a proof that no palette can fake them, tools that keep the results readable, and a sneaker that wears the idea on its two feet. None of this makes software faster. What designers gain is a new kind of constraint, together with a reason for it that anyone can check, on a laptop or on a quantum computer.

**Availability.** The website is at `https://qontext.qubitmen.workers.dev`. The source, including the scripts that compute this paper's numbers and draw its figures from the engine, is at `https://github.com/QubitMan-hub/Qontext`.

## References

[1] Simon Kochen and Ernst P. Specker. The problem of hidden variables in quantum mechanics. *Journal of Mathematics and Mechanics*, 17(1):59–87, 1967.

[2] John S. Bell. On the problem of hidden variables in quantum mechanics. *Reviews of Modern Physics*, 38:447–452, 1966. doi: 10.1103/RevModPhys.38.447.

[3] Asher Peres. Incompatible results of quantum measurements. *Physics Letters A*, 151: 107–108, 1990. doi: 10.1016/0375-9601(90)90172-K.

[4] N. David Mermin. Simple unified form for the major no-hidden-variables theorems. *Physical Review Letters*, 65:3373–3376, 1990. doi: 10.1103/PhysRevLett.65.3373.

[5] Costantino Budroni, Adán Cabello, Otfried Gühne, Matthias Kleinmann, and Jan-Åke Larsson. Kochen-Specker contextuality. *Reviews of Modern Physics*, 94:045007, 2022. doi: 10.1103/RevModPhys.94.045007.

[6] G. Kirchmair, F. Zähringer, R. Gerritsma, M. Kleinmann, O. Gühne, A. Cabello, R. Blatt, and C. F. Roos. State-independent experimental test of quantum contextuality. *Nature*, 460:494–497, 2009. doi: 10.1038/nature08172.

[7] A. Dikme, N. Reichel, A. Laghaout, and G. Björk. Measuring the Mermin-Peres magic square using an online quantum computer, 2020.

[8] Jia-Min Xu, Yi-Zheng Zhen, Yu-Xiang Yang, Zi-Mo Cheng, Zhi-Cheng Ren, Kai Chen, Xi-Lin Wang, and Hui-Tian Wang. Experimental demonstration of quantum pseudotelepathy, 2022.

[9] Alexander A. Klyachko, M. Ali Can, Sinem Binicioğlu, and Alexander S. Shumovsky. Simple test for hidden variables in spin-1 systems. *Physical Review Letters*, 101:020403, 2008. doi: 10.1103/PhysRevLett.101.020403.

[10] John F. Clauser, Michael A. Horne, Abner Shimony, and Richard A. Holt. Proposed experiment to test local hidden-variable theories. *Physical Review Letters*, 23:880–884, 1969. doi: 10.1103/PhysRevLett.23.880.

[11] Boris S. Cirel'son. Quantum generalizations of Bell's inequality. *Letters in Mathematical Physics*, 4:93–100, 1980. doi: 10.1007/BF00417500.

[12] Asher Peres. Two simple proofs of the Kochen-Specker theorem. *Journal of Physics A: Mathematical and General*, 24:L175–L178, 1991. doi: 10.1088/0305-4470/24/4/003.

[13] Adán Cabello, José M. Estebaranz, and Guillermo García-Alcaine. Bell-Kochen-Specker theorem: A proof with 18 vectors. *Physics Letters A*, 212:183–187, 1996. doi: 10.1016/0375-9601(96)00134-X.

[14] Sixia Yu and C. H. Oh. State-independent proof of Kochen-Specker theorem with 13 rays. *Physical Review Letters*, 108:030402, 2012. doi: 10.1103/PhysRevLett.108.030402.

[15] Björn Ottosson. A perceptual color space for image processing. https://bottosson.github.io/posts/oklab/, 2020.

[16] W3C. CSS color module level 4. W3C Candidate Recommendation, https://www.w3.org/TR/css-color-4/, 2024.

[17] Johannes Itten. *The Art of Color*. Reinhold, 1961.

[18] W3C. Web content accessibility guidelines (WCAG) 2.2. W3C Recommendation, https://www.w3.org/TR/WCAG22/, 2023.

[19] Andrew Somers. APCA: The accessible perceptual contrast algorithm. https://github.com/Myndex/apca-w3, 2022.

[20] W3C Design Tokens Community Group. Design tokens format module. Draft community group report, https://tr.designtokens.org/format/, 2025.

[21] Eduardo Reck Miranda, editor. *Quantum Computer Music: Foundations, Methods and Advanced Concepts*. Springer, 2022.

[22] Arianna Crippa, Yahui Chai, Omar Costa Hamido, Paulo Itaboraí, and Karl Jansen. Quantum computing inspired paintings: Reinterpreting classical masterpieces, 2024.

[23] James R. Wootton. Procedural generation using quantum computation. In *Proceedings of the International Conference on the Foundations of Digital Games (FDG)*, 2020.

[24] João S. Ferreira, Arianna Crippa, Astryd Park, Daniel Bultrini, Pierre Fromholz, Roman Lipski, Karl Jansen, and James R. Wootton. Quantum brush: A quantum computing-based tool for digital painting, 2025.

[25] Maxim Gumin. WaveFunctionCollapse. https://github.com/mxgmn/WaveFunctionCollapse, 2016.